42

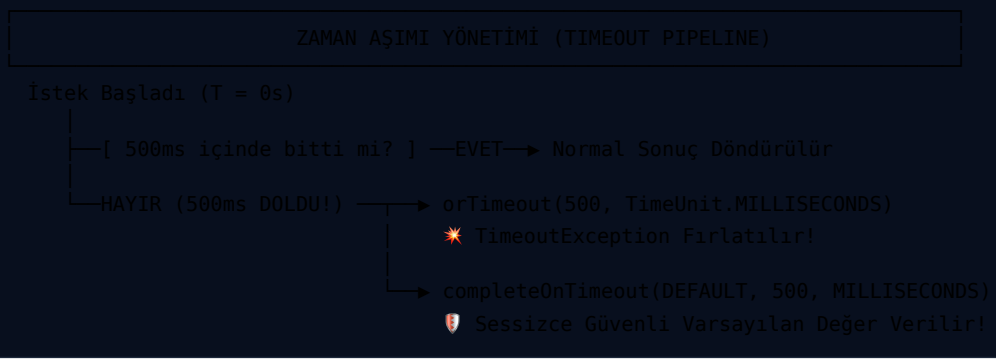
BÖLÜM: ASENKRON BORU HATLARI & COMPLETABLEFUTURE | MODULE: ASYNCHRONOUS PIPELINES & COMPLETABLEFUTURE

Zaman Aşımı ve Varsayılanlar: orTimeout() vs completeOnTimeout() ile Gecikme Yönetimi

Timeouts & Default Fallbacks: orTimeout() and completeOnTimeout()

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir restoranda sipariş verdiniz ve garsona dediniz ki: 1) orTimeout: '15 dakikada yemeğim gelmezse masayı terk eder, bağırarak çıkarım!' (Süre dolunca ortalığı ayağa kaldıran bir TimeoutException patlar). 2) completeOnTimeout: '15 dakikada biftek pişmezse bana hazır olan soğuk sandviçi getirin, karnımı doyurayım' demektir (Süre dolunca kimse üzülmez, önceden belirlenen yedek sandviç servis edilir).

🇬🇧 ENGLISH (ELI5)

`orTimeout()` is hard SLA enforcement: 'If data does not arrive in 200ms, abort the mission and throw a `TimeoutException` immediately!' `completeOnTimeout()` is graceful degradation: 'If the recommendation AI does not respond within 50ms, return a default list of trending items so the homepage never loads blank'.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Java 9+ ve Java 25 Zaman Aşımı Mimarisi: Java 8'de CompletableFuture için zaman aşımı desteği yoktu; harici bir `ScheduledExecutorService` ile geleceğe cancel/complete basmak gerekiyordu. Modern Java iki güçlü yöntem sunar: 1) `orTimeout(long timeout, TimeUnit unit)`: Süre aşıldığında future'ı `java.util.concurrent.TimeoutException` ile anormal şekilde tamamlar (`completeExceptionally`). 2) `completeOnTimeout(T value, long timeout, TimeUnit unit)`: Belirtilen süre dolduğunda eğer future henüz tamamlanmamışsa, verilen `value` ile normal olarak tamamlanır (`complete(value)`). Bu mekanizma arka planda HotSpot'un paylaşımlı `Delayer` zamanlayıcı thread'ini kullanır; her talep için yeni thread açmaz.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Java 9+ Timeout Architecture: Prior to Java 9, developers had to manually schedule cancellation tasks via `ScheduledExecutorService`. Modern CompletableFuture incorporates: 1) `orTimeout()`: If incomplete when duration elapses, it forces exceptional completion with `TimeoutException`. 2) `completeOnTimeout()`: If incomplete upon timer expiration, it transitions the future to normal completion using the designated fallback object. Both methods register non-blocking delays using an internal daemon `CompletableFuture.Delayer` wheel timer, avoiding per-timeout thread allocations.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: completeOnTimeout ile MAANG Servislerinde Graceful Degradation
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.TimeUnit;

public class TimeoutManagementDemo {
    record Recommendation(String productId, String category) {}

    static CompletableFuture<Recommendation> fetchAiRecommendations() {
        return CompletableFuture.supplyAsync(() -> {
            try {
                // Yapay zeka servisinin 800ms geciktiğini simüle et
                Thread.sleep(800);
            } catch (InterruptedException _) {}
            return new Recommendation("PROD-AI-99", "Personalized");
        });
    }

    void main() {
        Recommendation defaultFallback = new Recommendation("PROD-POPULAR-01", "Trending Now");

        // 200 ms SLA: 200 ms içinde gelmezse defaultFallback değerini bas!
        CompletableFuture<Recommendation> recoFuture = fetchAiRecommendations()
            .completeOnTimeout(defaultFallback, 200, TimeUnit.MILLISECONDS)
            .thenApply(reco -> {
                System.out.println("Kullanıcıya Gösterilen Ürün: " + reco);
                return reco;
            });

        Recommendation result = recoFuture.join();
        System.out.println("Ekrana Basılan Nihai Sonuç: " + result);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

completeOnTimeout çağrısı arka planda geciken asenkron işlemi İPTAL ETMEZ! Arka plandaki işçi thread geciken HTTP çağrısını yapmaya devam edebilir. Gerçekten soket veya thread iptali gerekiyorsa future.cancel(true) veya HTTP client zaman aşımları yapılandırılmalıdır.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

`completeOnTimeout` does NOT cancel the underlying upstream task! The worker thread running the database or HTTP call continues until completion in the background. To abort remote sockets, configure client-level timeouts alongside CompletableFuture.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- orTimeout süre dolduğunda TimeoutException fırlatır. completeOnTimeout ise sessizce yedek bir varsayılan değerle aşamayı tamamlar.

🎯 Key Takeaways for the Interview

- `orTimeout` aborts exceptionally with `TimeoutException`. `completeOnTimeout` silently injects a safe fallback upon SLA breach.